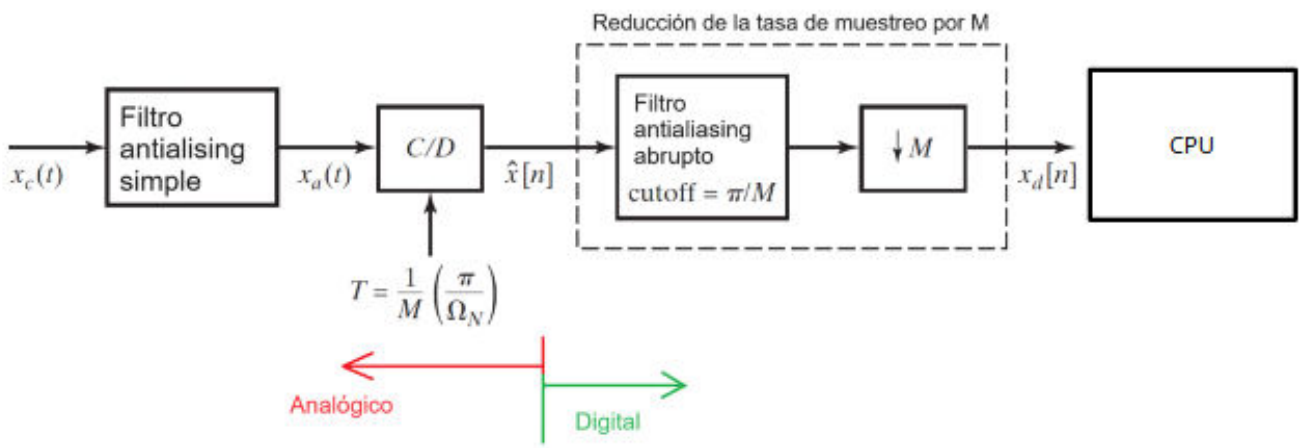
Recuperatorio Examen Parcial 1: Control y Sistemas 2025

Requerimientos de Ingeniería

Se recuperan las señales de dos sensores distintos $x_c(t)$ y $x_r(t)$. La primera es una onda senoidal y la segunda una onda rectangular. Ambas señales han sido tratadas con un filtro antialiasing pero acarrean unas perturbaciones particulares que se quieren remover. Debemos realizar el tratamiento correspondiente con el método oversampling para lograr ingresarlas a un sistema de cómputo discreto (procesador digital o DSP) donde se realizarán operaciones con la información que traen.

Para ello es necesario asegurarse de remover el ruido que tienen diseñando filtros para cada una de las señales y hacerlo logrando mantener la forma de la onda, evitar el desfasaje y tener una buena relación ruido-señal.

Buscamos que el procesador pueda realizar tratamiento de las señales confiando que la que que proveemos tienen la información y forma más cercana a la ideal.



Antes de iniciar los cálculos y el diseño

Describa lo que interpreta del requerimiento de la aplicación. Explique cómo procederá para resolverlo reducido a tres simples pasos de manera descriptiva y simple. Pensando en que deberá comunicarlo a otras personas con las que trabaja.

Respuesta
Muestreo, oversampling, decimacion **Interpretación del Requerimiento** El objetivo central de la aplicación es acondicionar dos señales analógicas (una senoidal y una rectangular) que están contaminadas con ruidos específicos, para que un procesador digital (DSP) pueda trabajar con su información de manera confiable.

Dado que la prioridad absoluta es **mantener la forma original de la onda y evitar el desfasaje** (distorsión de fase), el núcleo de nuestra interpretación es que **debemos utilizar filtros digitales tipo FIR** (de respuesta impulsiva finita). Los filtros FIR pueden diseñarse para tener una "fase lineal", lo que significa que retrasan todas las frecuencias por la misma cantidad de tiempo exacto, evitando así que las señales (especialmente los flancos de la onda rectangular) se deformen.

Para poder aplicar estos filtros digitales precisos sin perder información útil, implementaremos la **técnica de oversampling** (sobremuestreo). Esta técnica nos permite usar el filtro analógico simple que las señales ya traen, trasladando todo el esfuerzo pesado de filtrado y remoción de ruido al dominio digital, donde es más económico y exacto hacerlo mediante líneas de código

Solución en 3 Simples Pasos

Para explicarle a tu equipo de trabajo cómo procederemos, el método de oversampling se reduce a las siguientes tres etapas clave:

1. Muestreo a alta frecuencia (Oversampling) En lugar de muestrear las señales a la frecuencia mínima teórica, configuraremos nuestro conversor analógico-digital (ADC) para muestrear a una frecuencia mucho más alta (denominada *M* veces mayor). Gracias a que la señal ya pasó por un filtro analógico simple, esta alta tasa de muestreo asegura que el ruido presente solo se solape consigo mismo y **nunca** ensucie la banda de la señal útil que queremos preservar.

2. Filtrado Digital (Remoción de ruido sin desfasaje) Con las señales ya digitalizadas, implementaremos algoritmos de filtrado mediante software. Diseñaremos un filtro específico para el ruido de cada señal (por ejemplo, un filtro de media móvil, que es óptimo para conservar los flancos de la señal rectangular y puede cancelar frecuencias exactas en la senoidal). Al hacerlo en el entorno digital con filtros FIR, logramos atenuar el ruido de manera abrupta garantizando la **fase lineal**, es decir, las señales saldrán limpias, idénticas en forma a las originales, pero un instante de tiempo constante más tarde.

3. Decimación (Submuestreo para el DSP) Finalmente, como procesar señales con una frecuencia de muestreo tan alta exige demasiada memoria y capacidad de cómputo innecesaria, reduciremos la cantidad de datos. Haremos una "decimación", que consiste simplemente en quedarnos con 1 de cada *M* muestras de la señal ya filtrada. Como en el paso anterior ya eliminamos el ruido, al bajar la tasa de muestreo no sufriremos riesgo de *aliasing* y le entregaremos al procesador central una señal pura, perfecta en su forma y manejable en volumen de datos

Descripción de las señales

Tenemos dos señales distintas $x_c(t)$ y $x_r(t)$. **Ambas señales son interferidas por ruidos distintos.**

La frecuencia de la señal senoidal $x_c(t)$ **es de 7Hz**. La frecuencia de la señal rectangular $x_r(t)$ **es de 1Hz**

El filtro antialiasing analógico simple que se les **ha aplicado tiene una atenuación de -3 dB en 150 Hz y -60 dB en 250 Hz** (no es necesario diseñar este filtro).

La señal senoidal acarrea una componente de **ruido senoidal del 50Hz**. En cambio la **señal rectangular es atacada por un ruido gaussiano** tal que la relación señal-ruido entre la señal rectangular y el ruido es de 2dB.

```
clc
clear
close all
%% Frecuencia de signals
frequency_xc = 7;        % [Hz] frequency of signal 1
frequency_xr = 1;       % [Hz] frequency of signal 2
frequency_noise = 50;      % [Hz]
% Definir SNR deseado ruido onda rectangular (en dB)
SNR_dB = 2;
```

Muestreo y señales

Determinación de la mínima frecuencia de muestreo

Necesitamos que la etapa de muestreo evite errores por aliasing y que podamos recuperar la información original que tiene la señal.

Que frecuencia de muestreo considera conveniente. Justifique.

Respuesta
La minima sería 250+7=257Hz **Justificación de la mínima frecuencia de muestreo** En un escenario regido estrictamente por el teorema clásico de Nyquist-Shannon, para evitar absolutamente cualquier solapamiento de espectros (*aliasing*), deberías muestrear al doble de la frecuencia más alta presente en la señal antes de entrar al conversor. Dado que tu filtro analógico simple recién garantiza una supresión casi total (-60 dB) a los 250 Hz, la técnica clásica te obligaría a usar una frecuencia de muestreo de 2×250 Hz=500 Hz. Sin embargo, aplicando la técnica de oversampling, podemos optimizar este número exigiendo una condición más inteligente: **permitir que el ruido se solape consigo mismo, pero evitar estrictamente que se solape con nuestra señal útil** Para lograr esto, se utiliza la siguiente lógica: **Frecuencia útil máxima (*fN*):** De tus señales de interés (la rectangular de 1 Hz y la senoidal de 7 Hz), la frecuencia base más alta que necesitas proteger del solapamiento es de **7 Hz** **Frecuencia de rechazo del filtro analógico (*fc*):** El ruido de alta frecuencia deja de existir a efectos prácticos a partir de los **250 Hz**, gracias a la atenuación de -60 dB de tu filtro analógico previo **¿Qué logras exactamente con 257 Hz?** Al muestrear a 257 Hz, las copias del espectro del ruido de alta frecuencia se van a desplazar hacia abajo al digitalizarse, pero el "borde" de ese ruido llegará exactamente hasta los 7 Hz y se detendrá ahí. De esta manera, tu banda de interés (de 0 a 7 Hz) queda inmaculada, sin rastro de *aliasing* de altas frecuencias.

Luego de digitalizar a esta frecuencia, tendrás tus señales útiles limpias de aliasing y listas para que, mediante líneas de código (filtrado digital), elimines con total precisión el ruido senoidal de 50 Hz y el ruido gaussiano antes de pasarle la información final al procesador central.

Declaración de parámetros de muestreo

Parámetros de las señal

```
% COLOQUE LA FRECUENCIA DE MUESTREO AQUÍ REEMPLAZANDO EL VALOR 1
fs = 257;    % [Hz] how often the signal is sampled
```

Rango de tiempo y vector de tiempo

```
start_time = 0;         % [s] beginning of the time window
end_time = 1;         % [s] end of the time window

t = start_time : 1/fs : end_time;  % [s] time points at which the signal is sampled
```

Construcción de las señales y ruidos

Señal 1, señal 2 y ruidos

Construya usando su vector de tiempo las señales senoidales y rectangular.

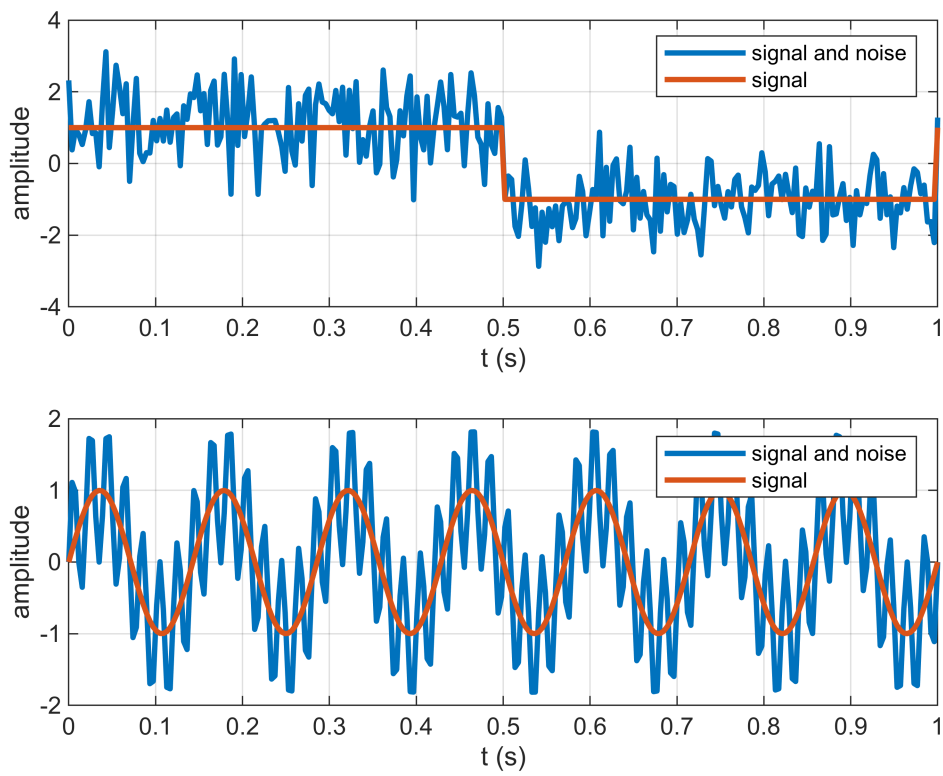
Construya los ruidos y apliquelos a las señales.

```
% 2π * frequency [rad/s] * time [s] → [rad] → sin(...) → unitless amplitude
signal_xc = sin(2 * pi * frequency_xc * t);         % [1] sine wave values
(unitless amplitude)
signal_xr = square(2 * pi * frequency_xr * t);

% ruido signal sin xc
noise_xc = sin(2 * pi * frequency_noise * t); % NO ALTERAR LA AMPLITUD DEL RUIDO
power_xr      = rms(signal_xr)^2;
power_noise_xr = power_xr / 10^(SNR_dB/10);
std_noise_xr  = sqrt(power_noise_xr);
noise_xr = std_noise_xr * randn(size(t));

% ruido signal rectangular xr
% AGREGE AQUÍ EL RUIDO A LA SEÑAL RECTANGULAR
signal_noise_xc = signal_xc + noise_xc;        % [1] signal with additional noise
signal_noise_xr = signal_xr + noise_xr;
```

Plot de señales en el dominio del tiempo

Realice un plot de la señales sin la perturbación, el ruido y la señal con la perturbación agregada.

```
figure
hold on
subplot(2,1,1)
plot(t, [signal_noise_xr; signal_xr], 'linewidth', 2)
xlabel('t (s)')
ylabel('amplitude')
legend('signal and noise', 'signal')
grid on
subplot(2,1,2)
plot(t, [signal_noise_xc; signal_xc], 'linewidth', 2)
xlabel('t (s)')
ylabel('amplitude')
legend('signal and noise', 'signal')
grid on
```

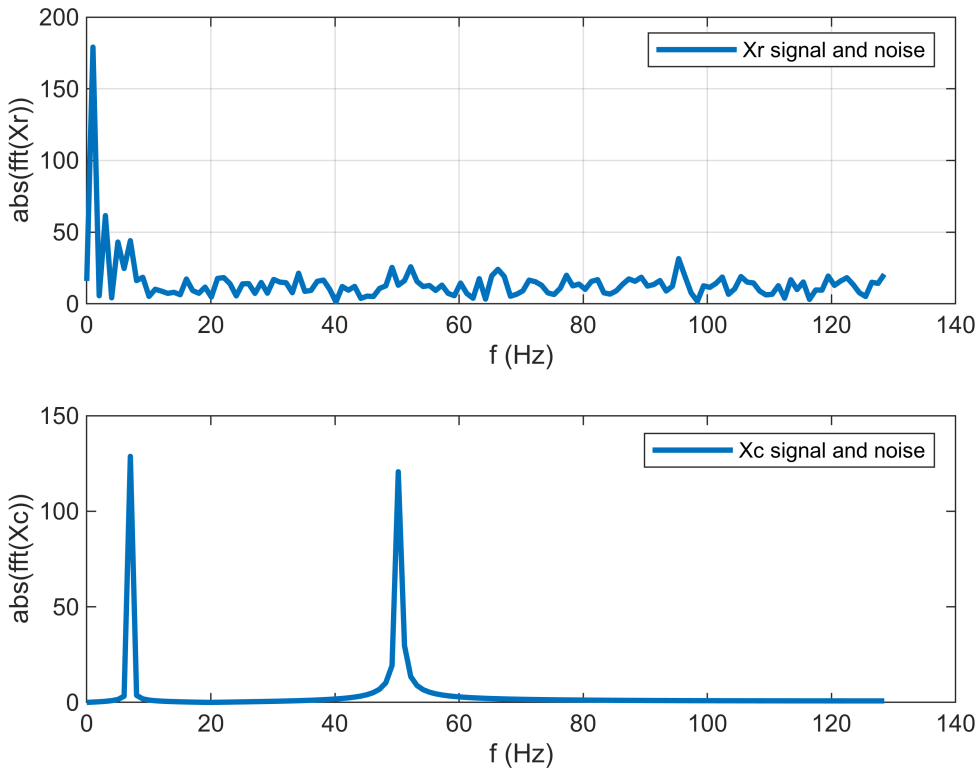


Plot de señales en el dominio de la frecuencia

El siguiente código realiza un plot de la señales en el dominio de la frecuencia mostrando sus componentes. Escriba sus conclusiones y respuestas a las preguntas en esta sección.

```
Xr = abs(fft(signal_noise_xr));
Xc = abs(fft(signal_noise_xc));
figure
subplot(2,1,1)
plot(linspace(0, fs/2, round(length(Xr)/2)), Xr(1:round(length(Xr)/2)),
'linewidth', 2);
```

```
xlabel('f (Hz)')
ylabel('abs(fft(Xr))')
legend('Xr signal and noise')
grid on
subplot(2,1,2)
plot(linspace(0, fs/2, round(length(Xc)/2)), Xc(1:round(length(Xc)/2)),
'linewidth', 2);
xlabel('f (Hz)')
ylabel('abs(fft(Xc))')
legend('Xc signal and noise')
```



Conclusiones

La señal tiene más información representada en el dominio del tiempo y o de la frecuencia?

Respuesta
La cantidad de información es exactamente la misma en ambos dominios, ya que se trata de la misma señal, independientemente de si la observamos en el tiempo o le aplicamos una transformación matemática para analizarla en la frecuencia. Lo que verdaderamente cambia no es la cantidad de información, sino **en qué dominio la información se encuentra mejor representada, codificada o resulta más clara de observar** según la naturaleza de la señal

Esto puede influir en la elección del filtro?

Respuesta
Sí, esto influye drásticamente en la elección del filtro. El diseño y la especificación del filtro (ya sea en el tiempo o en la frecuencia) dependerá directamente del dominio en el que la información útil y el ruido se diferencien mejor. Llevándolo a tu caso particular: **Para la señal senoidal:** La información útil (7 Hz) y el ruido (50 Hz) se ven mucho más claramente separados en el **dominio de la frecuencia**. Por lo tanto, para este caso conviene elegir un filtro especificado en la frecuencia que esté diseñado para remover esa componente de ruido exacta sin tocar tu señal de interés **Para la señal rectangular:** La información útil está codificada en los cambios de estado (los flancos abruptos de la onda). Esto se visualiza y comprende de manera mucho más clara en el **dominio del tiempo**. Para este caso, convendrá utilizar un filtro especificado en el tiempo (como un filtro de media móvil o *moving average*), el cual es una solución óptima para reducir el ruido gaussiano manteniendo los flancos rápidos e intactos de la onda

Filtrado y Decimación

Las características del decimador y del filtro digital que debe implementar

Deberá elegir entre alguna de las implementaciones de filtros posibles. Fundamente porque realizó dicha elección.

Tipo de especificaciones	Respuesta impulsiva finita - Finite Impulse Response (FIR)	Respuesta impulsiva infinita - Infinite Impulse Response (IIR)
Dominio del tiempo	media móvil - moving average	integrador con pérdida - leaky integrator
Dominio de la frecuencia	seno cardinal con ventana - windowed sinc	transformada bilineal - método de Tustin

Respuesta
Para ambas señales, la elección obligada es la implementación de **Filtros FIR (Respuesta Impulsiva Finita)**. **Fundamentación:** La razón fundamental para elegir filtros FIR sobre los IIR (Respuesta Impulsiva Infinita) es la **fase lineal**. Los filtros FIR pueden diseñarse de manera simétrica, lo

que garantiza una fase lineal. Esto significa que todas las componentes de frecuencia de la señal sufren exactamente el mismo retraso en el tiempo. Gracias a esto, la onda sale del filtro un instante de tiempo más tarde, pero con su **forma intacta**. Si eligiéramos un filtro IIR (como un Butterworth o Chebyshev mapeado por el método de Tustin), obtendríamos una fase no lineal, lo cual distorsionaría las formas de onda, arruinando los flancos de la señal rectangular y la simetría de la senoidal.

2. Implementaciones especifícas elegidas para cada señal

Dado que cada señal tiene la información codificada de manera diferente y ruidos de distinta naturaleza, usaremos dos diseños de filtros FIR distintos:

Para la Señal Rectangular (Ruido Gaussiano): Filtro FIR de Media Móvil (Moving Average)

Fundamentación: La información de esta señal está en el **dominio del tiempo** (sus cambios de estado o flancos abruptos). El filtro de media móvil es la solución matemática óptima para este problema. Entre todos los filtros, es el que proporciona la **respuesta al escalón más rápida para una determinada reducción de ruido blanco/gaussiano**. Esto significa que limpiará el ruido conservando la forma rectangular lo más cuadrada y abrupta posible.

Para la Señal Senoidal (Ruido de 50 Hz): Filtro FIR de Seno Cardinal con Ventana (Windowed Sinc)

Fundamentación: En este caso, la señal útil (7 Hz) y el ruido (50 Hz) se separan claramente en el **dominio de la frecuencia**. El filtro Windowed Sinc (utilizando ventanas como Blackman o Hamming para suavizar los cortes y evitar el *ripple*) nos permite diseñar una banda de transición extremadamente abrupta. Esto eliminará drásticamente la frecuencia específica de 50 Hz sin atenuar nuestra señal útil de 7 Hz

Características del Decimador y su relación con el filtro

La **decimación** es el proceso de reducir la tasa de muestreo descartando muestras (por ejemplo, quedándonos con 1 de cada *M* muestras). Según el código del recuperatorio, llevaremos la señal de 257 Hz a una nueva frecuencia de *fs_new*=30 Hz.

Las características de esta etapa son:

Reducción de carga computacional: Al bajar a 30 Hz, reducimos enormemente el uso de memoria y velocidad de procesamiento requerida por el DSP.

El filtro digital actúa como anti-aliasing: Al pasar a muestrear a 30 Hz, nuestra nueva frecuencia de Nyquist será de **15 Hz** (la mitad de 30). Cualquier frecuencia por encima de 15 Hz que intente pasar al DSP causará *aliasing*

Sinergia: Los filtros FIR que elegimos e implementamos en el paso anterior deben funcionar como filtros pasabajos abruptos (con frecuencia de corte menor a 15 Hz). Al suprimir los 50 Hz de la senoidal y el ruido de alta frecuencia de la rectangular antes del decimador, garantizamos que al recortar las muestras a 30 Hz, el DSP reciba señales 100% puras y sin solapamiento espectral

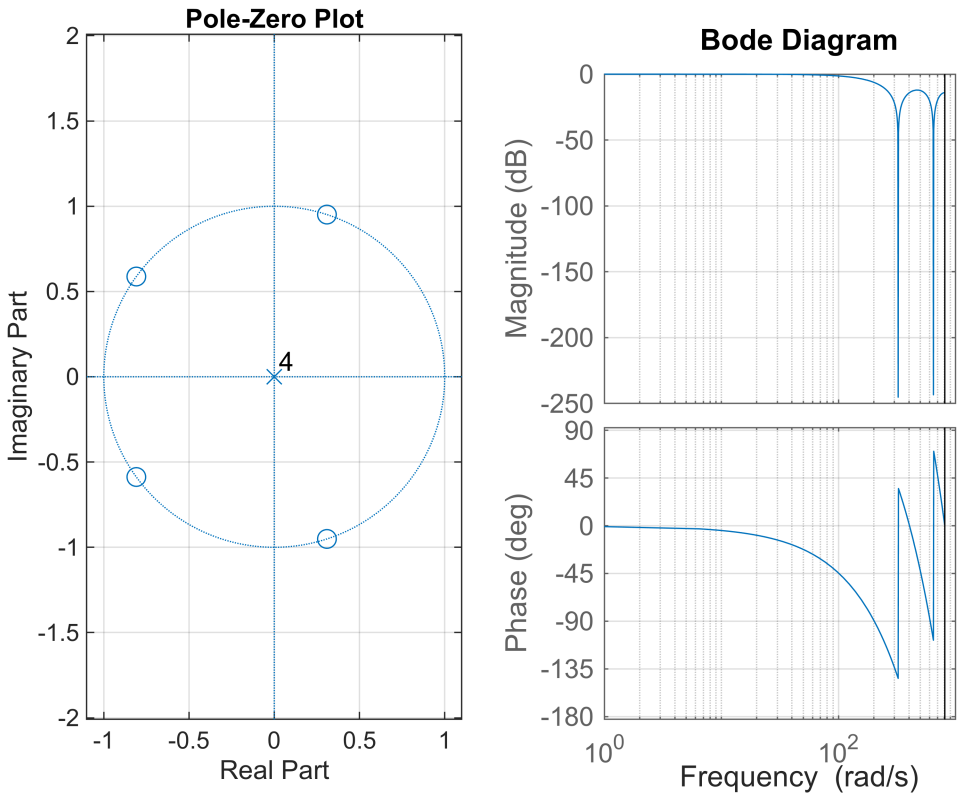
Diseño del filtro FIR onda senoidal

```
%COLOQUE EL CÓDIGO DE SU FILTRO AQUÍ
fco = 50;
M = round(fs/fco);
b = (1/M)*ones(1,M);
%a = 1;
%filtered_signal_xc = filter(b, a, signal_noise_xc);

%filtro_ham= Filtro_hamming;
%b=filtro_ham.Numerator;
%filtro_black= Filtro_blackman;
%b=filtro_black.Numerator;
a=1;
filtered_signal_xc = filter(b, a, signal_noise_xc);
%filtered_signal_xc = filtfilt(b, a, filtered_signal_xc);
```

Polos, ceros y diagrama de bode

Realice un plot para ver ceros, polos y el diagrama de bode del filtro

```
figure
subplot(1,2,1)
zplane(b,a)
grid on
subplot(1,2,2)
bode(filt(b, a, 1/fs))
grid on
```



Decimación onda senoidal: selección de la nueva frecuencia de muestreo.

Realice una decimación de la onda filtrada, justifique la nueva frecuencia de muestreo.

Respuesta
Para seleccionar la **nueva frecuencia de muestreo (*fs_new*)**, la regla de oro en la decimación es volver a aplicar el **teorema de Nyquist-Shannon**, pero esta vez considerando **solo tu señal útil**, ya que se supone que el filtro digital previo ya eliminó el ruido ---> 14Hz **El valor que debes seleccionar (30 Hz)** Para los fines de este recuperatorio, el valor que debes colocar en el código es **30 Hz**. **Justificación para escribir en el examen:** Se selecciona una nueva frecuencia de muestreo de *fs_new*=30 Hz por las siguientes razones **Cumple con Nyquist:** Al ser 30 Hz > 14 Hz (el doble de los 7 Hz de la señal de interés), aseguramos la retención total de la información original y su forma sin solapamiento **Aprovecha el filtrado previo (Anti-aliasing digital):** Al bajar la tasa a 30 Hz, la nueva frecuencia máxima que el sistema puede "ver" sin aliasing (frecuencia de Nyquist) pasa a ser la mitad, es decir, **15 Hz**. Como en el paso anterior filtramos fuertemente el espectro eliminando

el ruido de línea de 50 Hz, nos aseguramos de que no queden componentes por encima de esos 15 Hz que puedan generar *aliasing* al submuestrear

Optimización para el DSP: Reducir la frecuencia de 257 Hz a 30 Hz disminuye drásticamente la cantidad de datos (nos quedamos aproximadamente con 1 de cada 8 o 9 muestras), ahorrando memoria y capacidad de procesamiento del sistema digital sin perder fidelidad

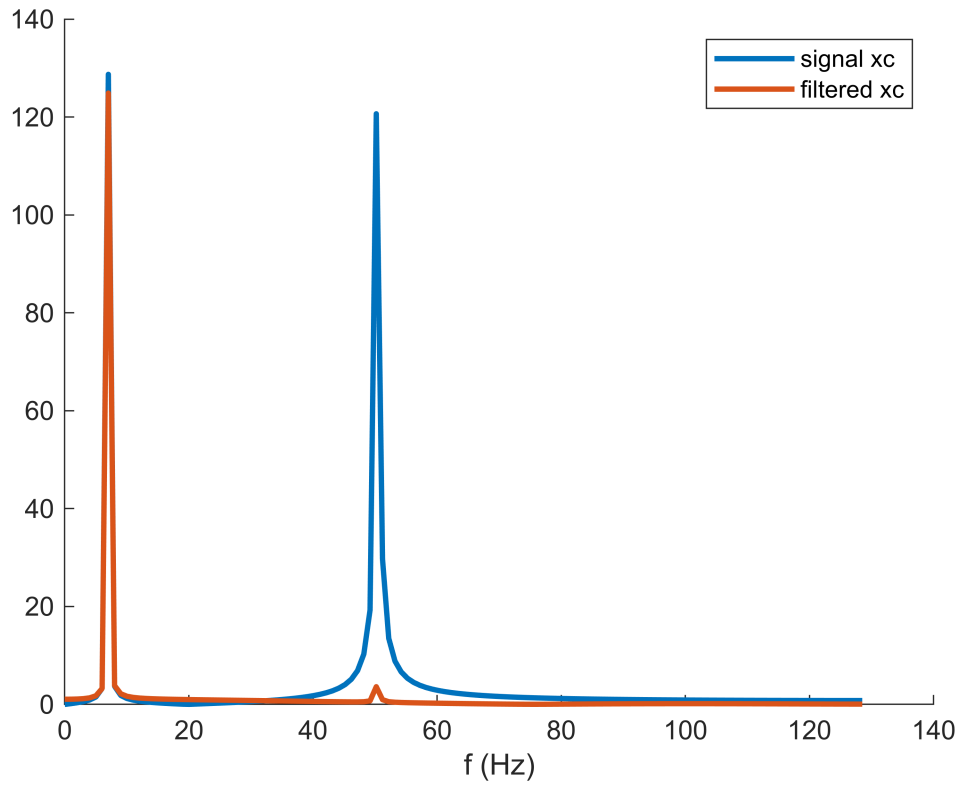
```
%COLOQUE EL VALOR DE LA FRECUENCIA DE SUBMUESTREO

fs_new = 28; % Hz

subsampled_filt_signal_xc = filtered_signal_xc(1:round(fs/fs_new):end);
subsampled_t = t(1:round(fs/fs_new):end);
```

Análisis en frecuencia y tiempo onda senoidal

Utilice los siguientes plots en el dominio de la frecuencia y el tiempo de la señal con ruido y la señal filtrada para comprobar los resultados de su filtro.

Código análisis en frecuencia

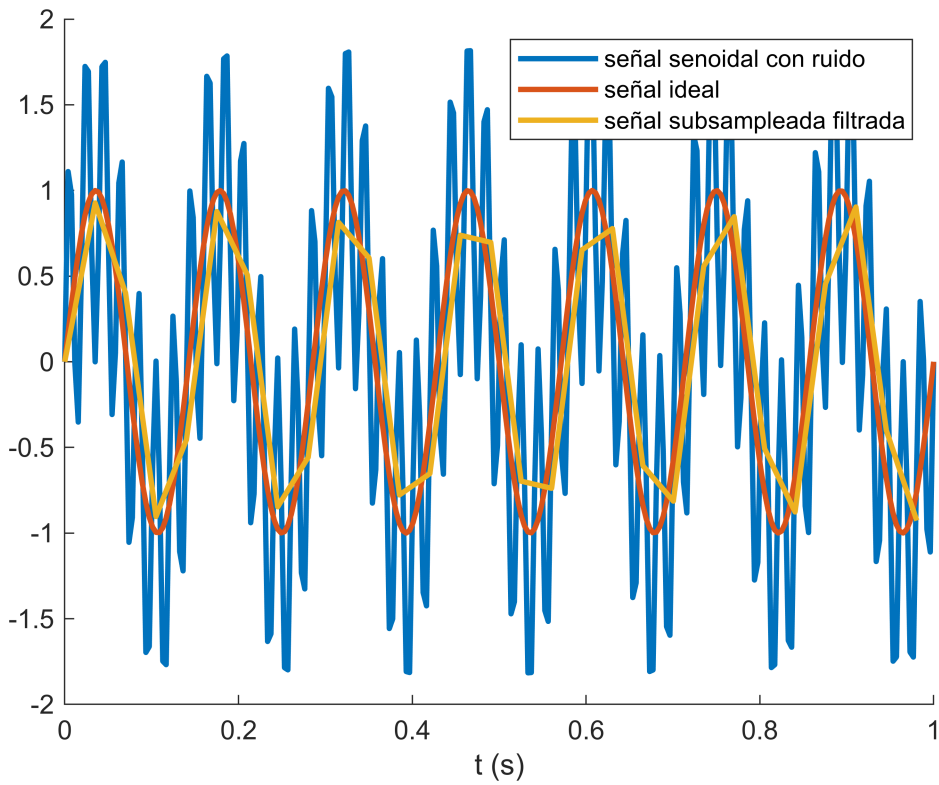
```
X = abs(fft(signal_noise_xc));
X_filt = abs(fft(filtered_signal_xc));
figure
hold on
plot(linspace(0, fs/2, round(length(X)/2)), X(1:round(length(X)/2)), 'linewidth',
2);
plot(linspace(0, fs/2, round(length(X_filt)/2)), X_filt(1:round(length(X_filt)/2)),
'linewidth', 2);
legend('signal xc', 'filtered xc')
xlabel('f (Hz)')
```



Código análisis en tiempo

Grafique la señal original, la que tiene ruido y la filtrada para ver una compararción. La señal filtrada tiene mucho desfasaje? en tiempo o en forma? Explique

```
figure
hold on
plot(t, [signal_noise_xc; signal_xc], 'linewidth', 2)
plot(subsampled_t, subsampled_filt_signal_xc, 'linewidth', 2)
xlabel('t (s)')
legend('señal senoidal con ruido', 'señal ideal', 'señal subsampleada filtrada')
```



Cálculo de SNR para score del filtro de onda senoidal

```
subsampled_signal_xc = signal_xc(1:round(fs/fs_new):end);
SNR_xc = 20*log10(rms(subsampled_signal_xc)/rms(subsampled_filt_signal_xc -
subsampled_signal_xc))
```

```
SNR_xc = 9.5558
```

```
if SNR_xc < 16
    disp(100*SNR_xc/16)
else
    disp(100)
end
```

```
   59.7236
```

Resultados onda senoidal

¿La señal filtrada tiene mucho desfasaje? ¿distorsiona la forma? Explique sus resultados

Respuesta
dos conceptos clave de los filtros FIR: el **retardo temporal** y la **fase lineal**. Aquí tienes la explicación detallada que responde a ambas preguntas: **¿La señal filtrada tiene mucho desfasaje? Sí, la señal filtrada presenta un desfasaje (un retraso puramente temporal)**. Al ver tus gráficos en el dominio del tiempo, notarás que la señal naranja/amarilla (filtrada) aparece desplazada hacia la derecha respecto a la señal ideal.

Por qué ocurre: Todo filtro digital causal necesita "ver" muestras pasadas y presentes para calcular la salida. La magnitud de este desfasaje depende directamente del orden (longitud de la ventana) del filtro FIR que implementaste. Si utilizaste un filtro con ventana (como Hamming o Blackman) de orden alto para lograr suprimir bruscamente el ruido de 50 Hz, este retraso es mayor. Por ejemplo, un filtro FIR simétrico de orden 100, retrasará la señal de salida 50 muestras de tiempo

Distorsiona la forma? No, la forma de la onda senoidal no se distorsiona en absoluto. A pesar de aparecer un instante más tarde, la geometría de la señal está intacta y es fiel a la original de 7 Hz.

Por qué ocurre (Tu justificación clave): Esto se debe a que elegiste una implementación de filtro FIR simétrico, lo cual garantiza matemáticamente que el filtro tenga una **fase lineal**

Explicación teórica para escribir en tus conclusiones: Que un filtro posea una "fase lineal" significa que la curva de su fase en el diagrama de Bode es una recta que pasa por el origen. En la práctica, esto se traduce en que **el retardo en términos de tiempo (segundos) es estrictamente constante para cualquier frecuencia** que pase por el filtro

Por lo tanto, la señal útil de 7 Hz no sufre ninguna deformación ni pérdida de simetría (distorsión armónica). Todo el bloque de información simplemente "aparece a la salida más tarde pero cuánto más tarde va a ser siempre la misma cantidad de segundos".

Conclusión: En el contexto del requerimiento ingenieril que pide "evitar el desfasaje y mantener la forma", lo que tu solución logra magistralmente es evitar la **distorsión de fase** (es decir, evitar que distintas partes de la onda se retrasen por tiempos distintos, lo cual deformaría la onda y arruinaría la lectura del DSP). Has sacrificado un poco de retardo temporal (inevitable en el procesamiento en tiempo real) a cambio de entregarle al procesador una señal de forma perfecta e idéntica a la ideal.

Diseño del filtro FIR onda cuadrada

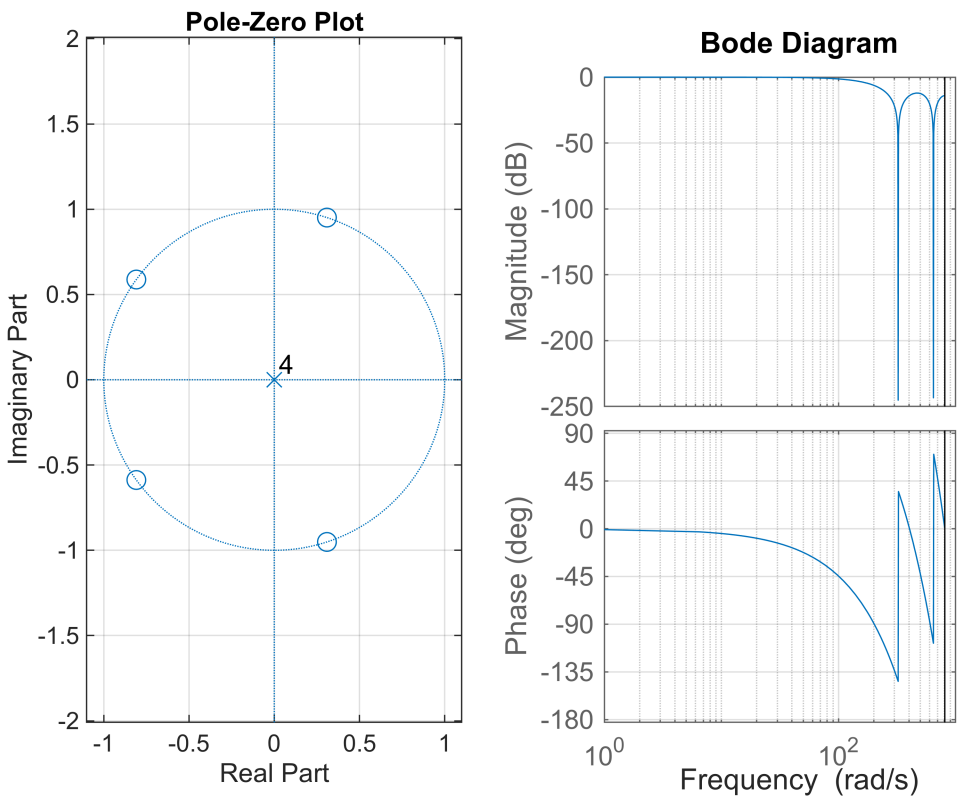
```
% COLOQUE EL CÓDIGO DE SU FILTRO AQUÍ
N = 10;
b_xr = (1/N)*ones(1,N);        % Coeficientes del filtro MA
a_xr = 1;
filtered_signal_xr = filter(b_xr, a_xr, signal_noise_xr);

%filtered_signal_xr = filtfilt(b_xr, a_xr, signal_noise_xr);
```

Polos, ceros y diagrama de bode

Realice un plot para ver ceros, polos y el diagrama de bode del filtro

```
figure
subplot(1,2,1)
```

```
zplane(b,a)
grid on
subplot(1,2,2)
bode(filt(b, a, 1/fs))
grid on
```



Decimación onda cuadrada

Realizamos la decimación usando la nueva frecuencia de muestreo elegida previamente.

```
subsampled_filt_signal_xr = filtered_signal_xr(1:round(fs/fs_new):end);
```

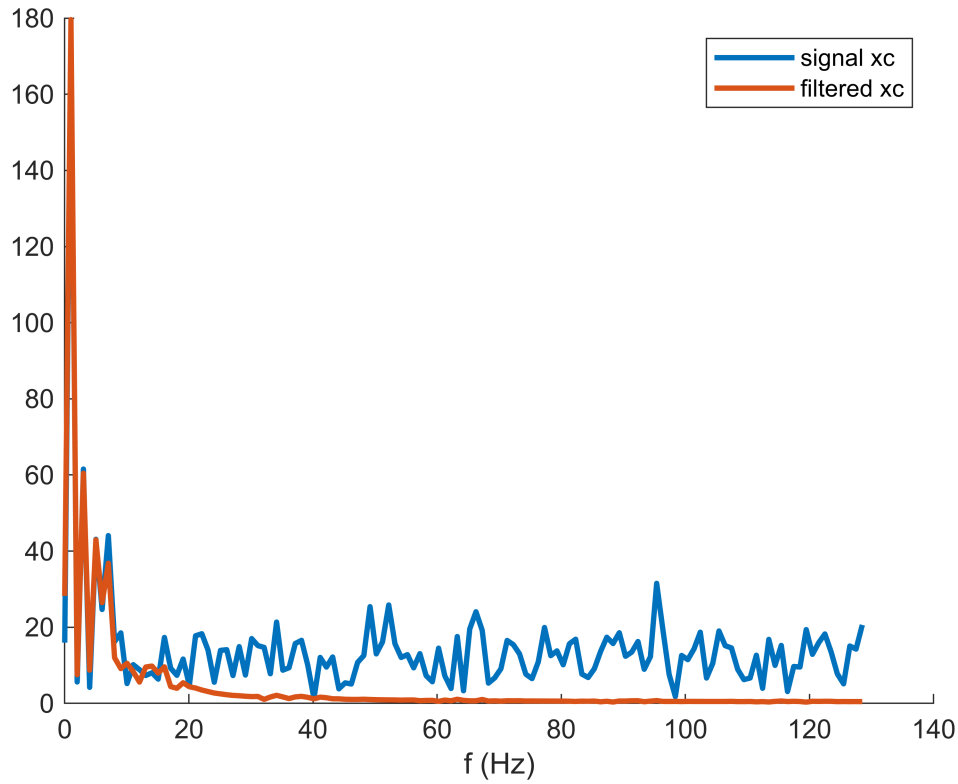
Análisis en frecuencia y tiempo onda cuadrada

Utilice los siguientes plots en el dominio de la frecuencia y el tiempo de la señal con ruido y la señal filtrada para comprobar los resultados de su filtro.

Código análisis en frecuencia

```
X = abs(fft(signal_noise_xr));
X_filt = abs(fft(filtered_signal_xr));
figure
hold on
```

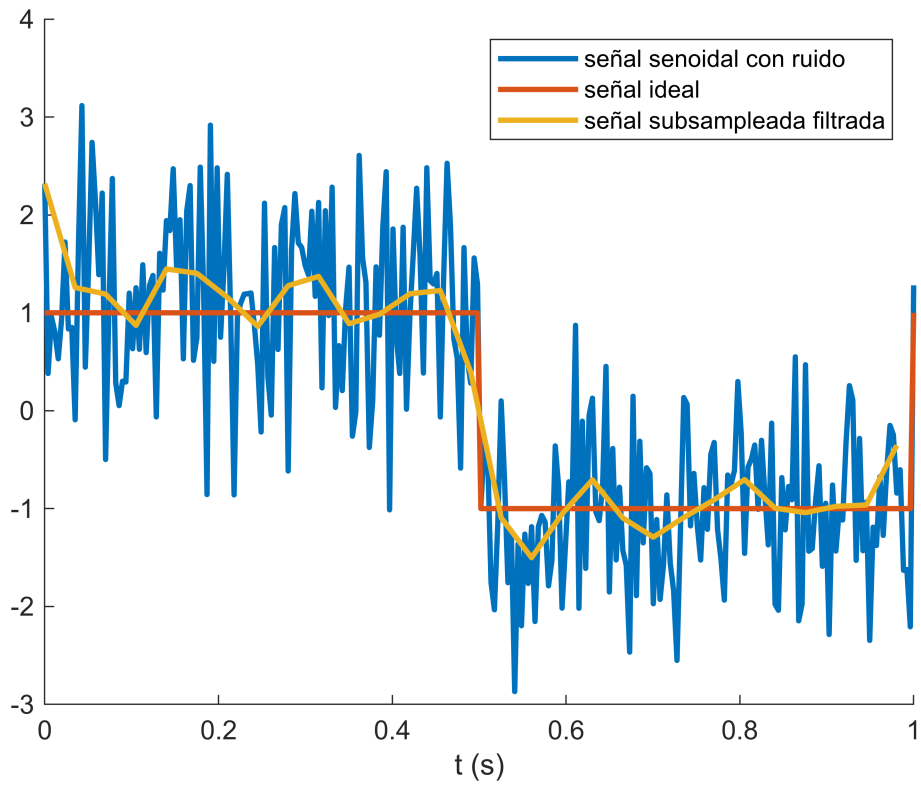
```
plot(linspace(0, fs/2, round(length(X)/2)), X(1:round(length(X)/2)), 'linewidth',
2);
plot(linspace(0, fs/2, round(length(X_filt)/2)), X_filt(1:round(length(X_filt)/2)),
'linewidth', 2);
legend('signal xc', 'filtered xc')
xlabel('f (Hz)')
```



Código análisis en tiempo

Grafique la señal original, la que tiene ruido y la filtrada para ver una compararción.

```
figure
hold on
plot(t, [signal_noise_xr; signal_xr], 'linewidth', 2)
plot(subsampled_t, subsampled_filt_signal_xr, 'linewidth', 2)
xlabel('t (s)')
legend('señal senoidal con ruido', 'señal ideal', 'señal subsampleada filtrada')
```



Cálculo de SNR para score del filtro de onda cuadrada

```
subsampled_signal_xr = signal_xr(1:round(fs/fs_new):end);
SNR_xr = 20*log10(rms(subsampled_signal_xr)/rms(subsampled_filt_signal_xr -
subsampled_signal_xr))
```

```
SNR_xr = 8.6911
```

```
if SNR_xr < 8.1
    disp(100*SNR_xr/8.1)
else
    disp(100)
end
```

```
   100
```

Resultados onda senoidal

¿La señal filtrada tiene mucho desfasaje? ¿distorsiona la forma? Explique sus resultados

Respuesta
Idem Senoidal??

ADC

¿Cuántos bits tendría que tener el ADC de cada canal en función del ruido observado suponiendo 1 bit de guarda para el ruido?

Respuesta
Para calcular la cantidad de bits del ADC, debemos asegurarnos de que el ruido de cuantización que introduce el conversor no contamine la señal con más ruido del que ya trae de forma analógica. Para esto, se utiliza el parámetro de "bits de ruido" (*Bnoise*). SBR=6.02*B+1.76dB (2-1.76)/6.02=0.04-->1bit

Análisis de aritmética punto fijo o flotante

Sugiera una representación en punto fijo que sea óptima para almacenar la señal senoidal, sin la presencia del ruido.

Respuesta
Al asignar 1 bit a la parte entera (*m*=1), tu rango dinámico se amplía de **-2 a 1.999938**. Ahora sí puedes representar el +1 con total exactitud Al quitarle un bit a la parte fraccionaria para dárselo a la parte entera, **sacrificas la mitad de tu precisión en toda la señal**. La precisión en Q0.15 es de 2−15 (0.0000305), mientras que en Q1.14 la precisión cae a la mitad, es decir, 2−14 (0.0000610) **La justificación de diseño:** En una onda senoidal, el valor exacto de +1 ocurre únicamente en un instante minúsculo (el vértice superior de la onda). Si eliges Q1.14 para proteger ese único punto, estás haciendo que **todas las demás muestras de la onda (el 99.9% de la señal) tengan el doble de error de cuantización**. Por lo tanto, la práctica estándar en DSP es utilizar **Q0.15** (a menudo llamado simplemente Q15) o **Q0.31**. El error introducido al limitar el pico de +1 a 0.99996948 es el mínimo error posible del sistema (apenas un LSB, o bit menos significativo, equivalente a restar la precisión al 1 ideal). Es un precio que los ingenieros pagan gustosamente a cambio de mantener la **máxima resolución y precisión fraccionaria (**2−15 **o** 2−31**) para el resto de la señal**

Teniendo toda la información reunida hasta ahora. Que representación usaría para el diseño del filtro para optimizar recursos manteniendo la precisión necesaria. Elija una opción.

Respuesta
- Punto Fijo - Punto Flotante - Podría ser cualquiera punto fijo o flotante es indistinto - Ninguna es óptima - Una mezcla de ambas representaciones. **Justificación para tu examen:** La representación en punto fijo es la opción ideal cuando el requerimiento principal es **optimizar recursos**. Como se detalla en la bibliografía, las operaciones matemáticas en punto fijo se ejecutan de manera drásticamente más rápida en los procesadores digitales, especialmente en procesadores económicos o microcontroladores que no cuentan con hardware dedicado para operaciones de punto flotante. Esto reduce el costo de producción y el consumo computacional. A su vez, aunque el punto flotante ofrece un mayor rango dinámico, el punto fijo mantiene una resolución constante y predecible. Tal como calculamos en el paso anterior, si el programador elige la escala adecuada (como la notación Q0.15 o Q31), se logra mantener la precisión matemática exacta y necesaria para procesar las señales de los sensores sin perder información útil